

Lab 5 Docker

Purpose: This lab aims to verify the configuration of Docker on your laptop

Install Docker.

<https://www.docker.com/products/docker-desktop/>

Configure a Node.js project

```
mkdir my-node-app  
cd my-node-app
```

Initialize a new project Node.js

```
npm init -y
```

Install Express

```
npm install express
```

Create a file app.js containing this:

```
const express = require('express');  
const app = express();  
const PORT = process.env.PORT || 3000;  
  
app.get('/', (req, res) => { res.send('Hello World!'); });  
  
app.listen(PORT, () => { console.log(`Server is running on port  
${PORT}`); });
```

Dockerfile

Create a Dockerfile in the project root directory containing the following content:

```
FROM node:lts

WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Create the docker image

```
docker build -t my-app-image .
```

Execute the container

```
docker run -d -p 3000:3000 -v /path/to/local/folder:/app my-app-image
```

Note: the `EXPOSE` command in the Dockerfile is for documentation purposes only: it indicates which port the application is going to use within the container, but it does not perform any actual opening port or port association. The actual connection between a host machine port and an internal container port is the responsibility of the container's execution tool, such as the `docker run` command or configuration via `docker-compose`. In case of a difference between the port specified in `EXPOSE` and the associated port during execution, the explicit configuration made by the runtime prevails. It is therefore necessary to ensure that the configured internal port actually matches the port on which the application is listening.

Docker Compose usage

Create a file `docker-compose.yml` to define and start the project service:

```
services:
  web:
    container_name: my-application
    build:
      context: .
      dockerfile: ./Dockerfile
    ports:
      - "9000:3000"
    volumes:
      - ./usr/src/app
    command: node app.js
```

Note: In real projects, the Dockerfile might be in a different subdirectory than the build context. Explicitly specifying `context` and `dockerfile`, although not always strictly necessary as in this case where default values are used, allows you to control precisely which directory is sent to the Docker daemon as the build context (`context`) and which file is used as the image definition (`dockerfile`). The build context determines which files and directories are accessible during the image build phase.

To build the Docker image of the application and launch compose, from the project directory where the yaml file is located, type:

```
docker-compose up --build
```

This command will build the Docker image, creating the container and booting the server. The application will be accessible at the URL <http://localhost:9000>.

Using environment variable

Edit the `docker-compose.yml` file to explicitly set the value of the `PORT` environment variable, which is already provided within the `app.js` file to determine the server startup port. The value to be assigned to the `PORT` variable should be 8080.

Also, update the port mapping in the `ports` section accordingly, so that port 8080 is used inside the container, but continues to be exposed externally on port 9000 on the host machine.

The required configuration is as follows:

```
services:
  web:
    container_name: my-application
    build:
      context: .
      dockerfile: ./Dockerfile
    ports:
      - "9000:8080"
    volumes:
      - ./usr/src/app
    environment:
      PORT: 8080
    command: node app.js
```

Note: In this exercise, we simply value an environment variable already provided in the code. Setting the `PORT` variable takes effect only because the application was developed to read its value at startup. In the absence of this reading in the code, simply valuing the variable would have no effect.

Usage of anchor YAML

Edit the docker-compose.yml file by introducing a YAML anchor to define the value of the port to be used within the container.

The PORT environment variable will need to be set using the defined anchor.

The port mapping in the ports section will have to be updated manually and consistently, since YAML anchors cannot be used in this section.

The required configuration is the following:

```
x-constants:
  port: &app_port 8080

services:
  web:
    container_name: my-application
    build:
      context: .
      dockerfile: ./Dockerfile
    ports:
      - "9000:8080"
    volumes:
      - ./usr/src/app
    environment:
      PORT: *app_port
    command: node app.js
```

Note: In YAML, anchors allow you to define reusable values within the file. However, you cannot use anchors directly in the ports section, which requires explicit strings.

Integrating a MYSQL database service in the compose

Extend the docker-compose.yml file by adding a new database service based on the official mysql:8 image.

Deliverables are:

- Use mysql:8 as the image.
- Configure the database via environment variables only (define YAML anchors at will), without using initialization files.
 - Set the user password using MYSQL_ROOT_PASSWORD variable.
 - Automatically create a new database by specifying the name using the MYSQL_DATABASE variable.
 - Create an application user configuring MYSQL_USER and the corresponding password using MYSQL_PASSWORD.
- Expose the port 3306 outside the container.
- Ensure that the service is started correctly and that the database and user are created at startup.

Note: The MySQL service exposes port 3306 by default. The official MySQL image supports automatic creation of the database and application user through configuration of environment variables, requiring no additional manual intervention.

Verify the MYSQL database service

Once the configuration of the MySQL service is complete, check for proper operation by performing the following checks.

Start compose with the command:

```
docker compose up --build
```

Verify that the database service is started correctly and that no connection or configuration errors appear in the logs.

Connect to the MySQL database using the `docker exec` command, e.g:

```
docker exec -it <database container name> mysql -u  
<user application name> -p
```

dove:

- `<database container name>` corresponds to the value specified in the `container_name` field of the database service within the `docker-compose.yml` file,
- `<user application name>` corresponds to the value of the `MYSQL_USER` variable.
- The option `-p` will prompt for the password, which must match the value set in the `MYSQL_PASSWORD` variable.

Once connected to the database, run the command:

```
SHOW DATABASES;
```

and verify that the database with the name specified in the environment variable exists `MYSQL_DATABASE`.

Alternatively, you can use a graphical client such as MySQL Workbench to connect to the MySQL database, specifying the localhost address, port 3306 and configured credentials.

Note: During the initial startup phase of the MySQL container, a warning message may appear indicating that the root user has been created temporarily without a password. This behavior is normal: during the database file initialization phase (`initialize-insecure`), the server starts without a password to allow the configuration script to later apply the password specified via the `MYSQL_ROOT_PASSWORD` variable. At the end of the `initialize-insecure` procedure, the root user will be correctly protected by the configured password. There is no need for manual intervention: this is expected behavior in the official MySQL image.